

Alineamiento de lecturas genómicas

k-mers como semillas: seis algoritmos, su análisis y su validación experimental

Kuilmer Santiago Hernández Cruz · Juan José Berrio Español · Santiago Serrano Cuellar
Análisis de Algoritmos (SIST0175) · Grupo 3

ADN

Buscar 50 letras con errores en un millón

Genoma sintético de n = 10^6 bases y lecturas de m = 50 con 2 % de sustituciones y 1 % de indels (semilla fija). Comparar cada lectura contra todo el genoma cuesta Θ(m·n); con semillas, solo Θ(m^2).

k-mers
2 bits por base, Θ(n)

Ordenar
C1 merge sort
C4 quicksort ·
C5 en disco

Índice
C2: posiciones por k-mer

Semillas
búsqueda binaria
+ votos

Alinear
C3 Smith-Waterman en la región

Paralelo
C6 MapReduce
p procesos

100 / 100
lecturas ubicadas a ≤ 3 bases de su posición real

156
pruebas automáticas (pytest), todas pasan

0
usos de sorted / heapq en el núcleo de los componentes

C1 Merge sort: Θ(n log n) y óptimo

Recurrencia leída del código

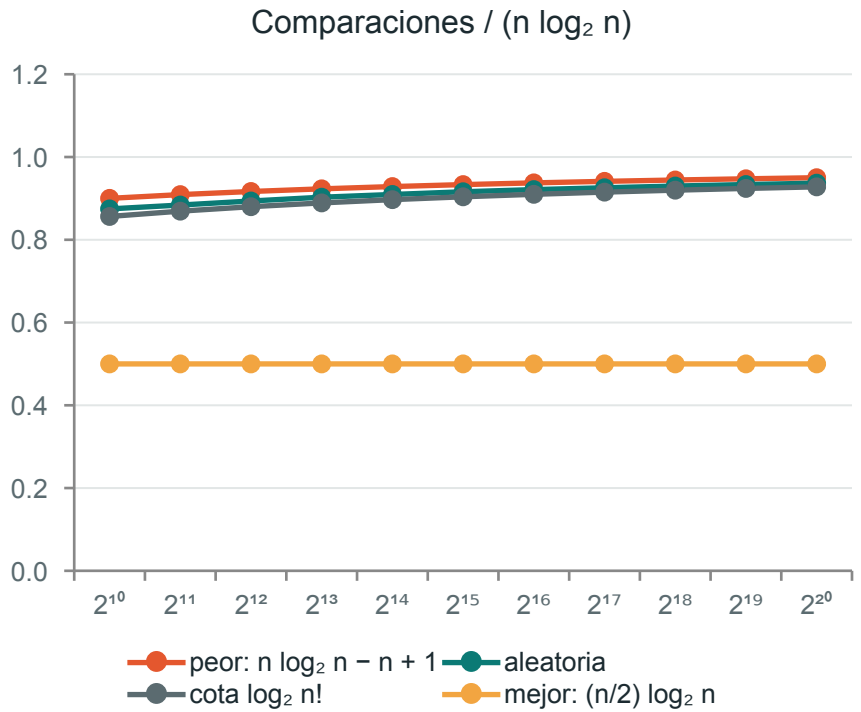
$T(n) = 2 \cdot T(n/2) + \Theta(n)$
Maestro: $a = 2$, $b = 2$, $f(n) = n = n^{(\log_2 2)} \rightarrow$ caso 2 $\rightarrow \Theta(n \log n)$

Cota inferior: árbol de decisión

$n!$ hojas $\Rightarrow$ altura $h \geq \log_2 n! = \Omega(n \log n)$. Ningún ordenamiento por comparaciones lo mejora.

1,12
pendiente log-log medida (teoría: 1,10)

exacto
mejor y peor caso de comparaciones



C4 Quicksort aleatorizado: 2n ln n esperado

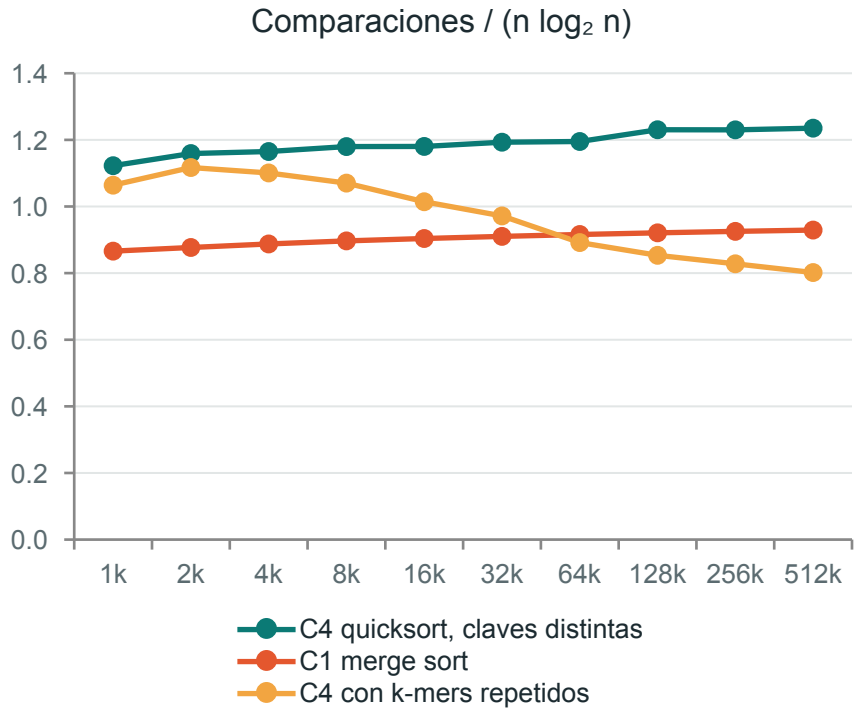
Por qué 2n ln n

z_i y z_j se comparan solo si uno es el primer pivote de $\{z_i \ldots z_j\}$: prob. $2/(j-i+1)$.
$E[C] = \Sigma\, 2/(j-i+1) = 2(n+1)H_n - 4n \approx 2n \ln n$

≈ 1,00
medidas / esperadas (0,99–1,03)

17 B
extra por elemento de merge sort; quicksort ≈ 0

Pivote aleatorio: ninguna entrada es mala de forma sistemática (también la ya ordenada). Partición de 3 vías: los k-mers repetidos se fijan de una vez.



C2 Arreglo dinámico: 3 créditos pagan todo

Método contable

Se cobran 3 créditos por append: 1 paga la escritura y 2 se guardan. Al duplicar de c a 2c, los c/2 elementos nuevos guardaron c créditos: justo lo que cuesta copiar c. El saldo nunca es negativo ⇒ costo ≤ 3n ⇒ O(1) amortizado.

< 3

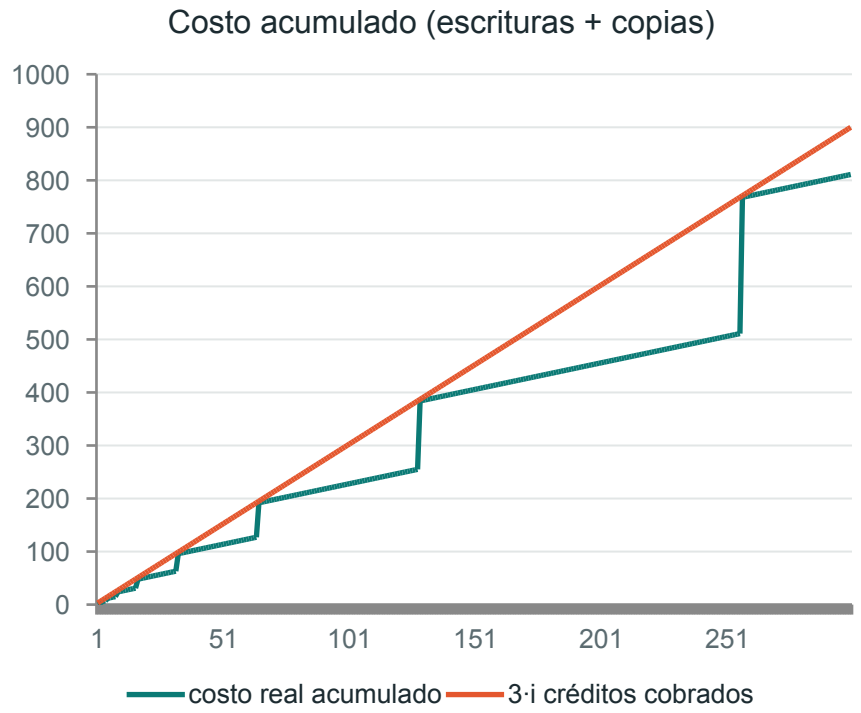
costo / n en todo n; casi 3 justo después de duplicar

0

saldo mínimo del banco de créditos

1,01

pendiente log-log del tiempo (teoría: 1)



C5 Ordenamiento externo: E/S exactas

$$E/S = 2 \cdot (N/B) \cdot (1 + \lceil \log_{M/B-1}(N/M) \rceil)$$

Corridas de M registros ordenadas con C4; mezcla de k = M/B − 1 corridas a la vez con un min-heap propio.

M (memoria)	k	corridas	pasadas	E/S contadas	E/S predichas
50.000	499	20	1	40.000	40.000
10.000	99	100	2	60.000	60.000
2.000	19	500	3	80.000	80.000
1.000	9	1.000	4	100.000	100.000
500	4	2.000	6	140.000	140.000

N = 10^6 registros de 12 bytes (kmer uint64 + posición uint32), B = 100. Todo acceso pasa por DiscoSimulado.leer_bloque / escribir_bloque.

10 / 10

configuraciones con E/S contadas = predichas

≤ M

pico de registros en memoria (instrumentado)

+20 000

E/S por cada pasada extra (= 2N/B)

C3 Smith-Waterman: tabla Θ(mn)

Recurrencia (+2 / −1 / hueco −2)

```
H[i][j] = max(0,
    H[i−1][j−1] + s(aᵢ, bⱼ),
    H[i−1][j] − 2,  H[i][j−1] − 2)
```

Por qué programación dinámica

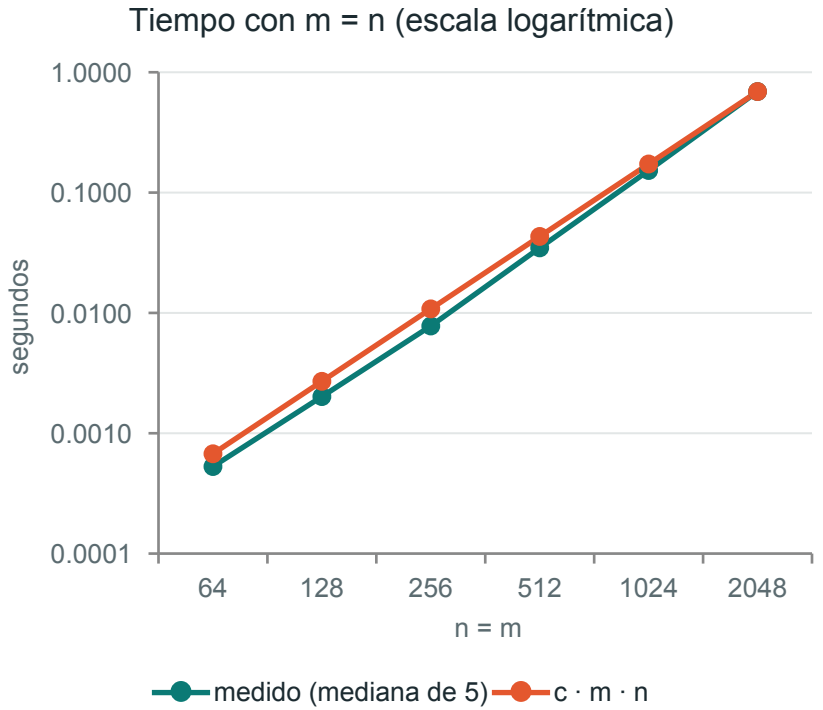
Subestructura óptima (cortar y pegar) y subproblemas superpuestos: recursión ingenua ≥ 3^min(m,n) llamadas; tabla de (m+1)(n+1) celdas.

2,08

pendiente con m = n (teoría: 2)

= m·n

celdas calculadas en toda entrada



C6 MapReduce: Amdahl explica el techo

Ley de Amdahl

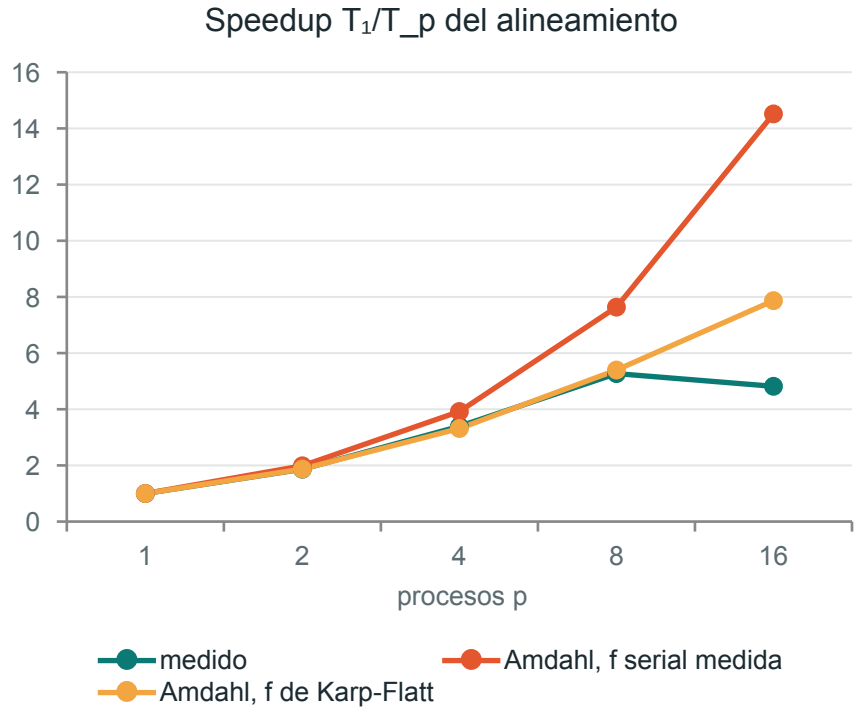
$S(p) = 1 / (f + (1 - f)/p) \leq 1/f$
map: p procesos alinean bloques de lecturas · reduce: unir en orden

5,27×
speedup con 8 procesos (Amdahl con f = 0,007: 7,64×)

f ≈ 0,07
fracción serial efectiva (Karp-Flatt), constante p = 2–8

2,18×
sistema completo: el índice serial fija 1/f ≈ 3,0

8 + 8
núcleos físicos + hilos SMT: de 8 a 16 no mejora



✓ Validación: la teoría predice el conteo

Comp.	Predicción	Medido	¿Coincide?
C1	Θ(n log n): pendiente 1,10; comparaciones exactas en mejor y peor caso	pendiente 1,12; conteos exactos	**sí**
C2	costo ≤ 3n; saldo ≥ 0; Θ(n)	costo/n < 3; saldo mín. 0; pendiente 1,01	**sí**
C3	Θ(mn): pendiente 1 (m fija) y 2 (m = n)	1,02 y 2,08; celdas = m·n	**sí**
C4	$E[C] = 2(n+1)H_n - 4n$	0,99–1,03 veces lo esperado; pendiente 1,14	**sí**
C5	E/S = 2(N/B)(1 + ⌈log_k(N/M)⌉)	10 de 10 iguales; memoria ≤ M	**sí**
C6	Amdahl con f medida	5,3× vs 7,6× con p = 8; con f de Karp-Flatt ±3 %	**parcial**

Protocolo: perf_counter, 5 repeticiones, mediana y Q1–Q3, datos fuera del tiempo medido, ≥ 5 tamaños en progresión geométrica, pendiente por mínimos cuadrados en log-log.

≠ Discusión: dónde no coincide y por qué

Coincide

- Conteos exactos: comparaciones (C1, C4), créditos (C2), celdas (C3), E/S (C5)
- Pendientes log-log con error ≤ 0,08 en todas las hipótesis de complejidad
- La teoría predice cómo crece el costo, no los segundos

No coincide (y la causa)

- Intérprete: ≈ 140 ns por comparación, unas 100 veces C; cambia c, no la pendiente
- Caché: $T/(n \log n)$ sube 13 % con $n \geq 10^5$; SW con m = n da 2,08
- C6: copy-on-write del índice heredado por fork y SMT (8 núcleos reales)

Conclusiones

- Los seis componentes funcionan integrados: 100/100 lecturas ubicadas, 156 pruebas.
- Cada cota está respaldada por una prueba y por un experimento; los conteos son exactos.
- El único desajuste (speedup de C6) queda explicado con Karp-Flatt y Amdahl.

Santiago Serrano Cuellar

C1 merge sort · C4 quicksort

Juan José Berrio Español

C2 arreglo dinámico · C5 ordenamiento externo

Kuilmer Santiago Hernández Cruz

C3 Smith-Waterman · C6 MapReduce

¿Preguntas?